

# git ours and theirs

Which side wins a conflict

[blog.andreладemann.de](https://blog.andreладemann.de)

**In a rebase, “theirs” means your own commits.** `-Xours` and `-Xtheirs` point at opposite sides depending on whether you merge or rebase.

## YOU ARE ON BRANCH-A, THE ARGUMENT IS BRANCH-B

| Command                                    | ours is                                         | theirs is                                    |
|--------------------------------------------|-------------------------------------------------|----------------------------------------------|
| <code>git merge -X...<br/>branch-b</code>  | <b>branch-a</b> – the branch you<br>are on      | <b>branch-b</b> – the one<br>being merged in |
| <code>git rebase -X...<br/>branch-b</code> | <b>branch-b</b> – the branch you<br>replay onto | <b>branch-a</b> – your own<br>commits        |

## WHY IT IS INVERTED

**Rebase replays your commits onto the target** The target branch is what already exists – “ours” – and each replayed commit arrives as an incoming change, so it is “theirs”. The naming reflects the mechanics, not whose code it is.

## THE TWO YOU ACTUALLY NEED

```
# Rebasing your feature branch onto master, your branch wins
git rebase -Xtheirs master

# Merging master into your branch, your branch wins
git merge -Xours origin/master
```

Both say the same thing in plain language: keep my work on conflicts. Available since Git 1.7.3.

## BEFORE YOU REACH FOR IT

**It only decides conflicting hunks** Non-conflicting changes from both sides are still merged. `-X` is a strategy option, not a way to discard the other branch.

**It resolves silently** Use it when you already know one side is correct – not to make a wall of conflicts go away.

**Check what you got** Run `git diff` against the branch you expected to win before you push, and let the tests have the final word.